DOCUMENTACIÓN PROYECTO PRUEBAS SABER



NOMBRES Y APELLIDOS DE LOS ESTUDIANTES

JOHAN SANTIAGO LOPEZ ORTEGA

SEBASTIAN TINTINAGO PANTOJA

EDDY EMMANUEL SANCHEZ OBANDO

Profesor:

Julio Ariel Hurtado Alegria

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Departamento de Sistemas

Ingeniería del *Software* II

Popayán, octubre 2026

JOHAN SANTIAGO LOPEZ ORTEGA

SEBASTIAN TINTINAGO PANTOJA

EDDY EMMANUEL SANCHEZ OBANDO

DOCUMENTACIÓN DEL PROYECTO BANCO DE PREGUNTAS SABER PRO – PRIMER CORTE

Entregable del primer corte presentado en el curso de Ingeniería de Software II

Estudiantes del:

Programa de Ingeniería de Sistemas

Profesor:

Julio Ariel Hurtado Alegria

Popayán

2026

Contenido

Lista de Figuras iii
Lista de Tablas iv
Lista de Siglas v
Documentación del proyecto Banco de Preguntas Saber Pro 1
1.- Introducción 1
2.- Historias de usuario 2
2.1.- HU-01: Crear pregunta 2
2.2.- HU-02: Enviar pregunta a revisión 2
2.3.- HU-03: Listar mis preguntas 3
2.4.- HU-04: Asignar revisores 3
3.- Prototipos y prueba de usabilidad 4
3.1.- Prototipos 4
3.2.- Prueba de usabilidad 5
4.- Planificación del Sprint 1 7
5.- Escenario de calidad: modificabilidad 8
6.- Arquitectura 9
6.1.- Tipo de aplicación y tecnologías 9
6.2.- Diagramas C4 9
6.3.- ¿Por qué capas y MVC? 10
7.- Explicación del código 12
7.1.- Organización de los paquetes 12
7.2.- ¿Cómo arranca la aplicación? 12
7.3.- Clases del dominio 13
7.4.- HU-01: crear una pregunta paso a paso 13
7.5.- HU-02: enviar a revisión 14
7.6.- HU-03: filtros y paginación 14
7.7.- HU-04: asignar revisores 15
7.8.- Base de datos 15
7.9.- Contraseñas 16
8.- Principios SOLID y patrones de diseño 17
8.1.- Principios SOLID 17
8.2.- Patrones de diseño 17
8.2.1.- Observer 18

8.2.2.- Factory 18
9.- Pruebas unitarias 19
10.- Lo que falta y lo que vamos a mejorar 20
11.- Enlaces 21
Bibliografía 22

Lista de Figuras

Figura 3.1 Prototipo del formulario para crear una pregunta 4
Figura 3.2 Prototipo de la ventana «Mis preguntas» 5
Figura 3.3 Prototipo de la ventana «Asignar revisores» 5
Figura 4.1 Tablero de tareas del Sprint 1 7
Figura 6.1 Diagrama de contexto (C4 nivel 1) 9
Figura 6.2 Diagrama de componentes por capas (C4 nivel 3) 10

Lista de Tablas

Tabla 2.1 Historias de usuario del primer corte 2
Tabla 3.1 Tareas de la prueba de usabilidad 6
Tabla 4.1 Tareas del Sprint 1 7
Tabla 5.1 Escenario de calidad de modificabilidad 8
Tabla 6.1 Tecnologías usadas 9
Tabla 7.1 Paquetes del proyecto 12
Tabla 7.2 Tablas de la base de datos 16
Tabla 8.1 Patrones de diseño usados 17
Tabla 9.1 Resumen de las pruebas unitarias 19

Lista de Siglas

Sigla	Significado
BD	Base de datos
GUI	Interfaz gráfica de usuario
HU	Historia de usuario
MVC	Modelo-Vista-Controlador
RF	Requisito funcional
RNF	Requisito no funcional
SMTP	Protocolo para enviar correos
SOLID	Cinco principios de diseño orientado a objetos
SQL	Lenguaje de consultas para bases de datos

Documentación del proyecto Banco de Preguntas Saber Pro

1.- Introducción

En este documento explicamos cómo construimos la primera versión del sistema *Banco de Preguntas Saber Pro* para el curso de Ingeniería de Software II. La idea es que cualquier persona que lo lea entienda qué hace la aplicación, cómo está organizado el código y por qué tomamos las decisiones que tomamos.

Para este primer corte nos pidieron una aplicación de escritorio en Java con una arquitectura monolítica de tres capas, usando el patrón MVC, los principios SOLID y algunos patrones de diseño. Implementamos cuatro historias de usuario:

- Crear preguntas de selección múltiple con validación.
- Enviar las preguntas a revisión.
- Listar las preguntas del autor con filtros y páginas.
- Asignar revisores y avisarles por correo.

La aplicación también tiene el inicio de sesión y el registro de usuarios con roles, que hicimos en los talleres de laboratorio.

2.- Historias de usuario

La Tabla 2.1 resume las cuatro historias de usuario que implementamos. Después explicamos cada una con sus criterios de aceptación, es decir, lo que se tiene que cumplir para que la historia se dé por terminada.

Código	Rol	Historia
HU-01	Autor de preguntas	Crear preguntas de selección múltiple con única respuesta para alimentar el banco de preguntas Saber Pro.
HU-02	Autor de preguntas	Cambiar el estado de sus preguntas de «Borrador» a «Pendiente de revisión» para que el administrador les asigne revisores.
HU-03	Autor de preguntas	Listar las preguntas que ha creado, con filtros y paginación, para verlas y editarlas.
HU-04	Administrador	Asignar al menos un revisor a las preguntas «Pendiente de revisión»; el sistema les avisa por correo.

Tabla 2.1 Historias de usuario del primer corte

2.1.- HU-01: Crear pregunta

- El autor llena todos los campos: título, contexto, pregunta directa, las cuatro opciones (A, B, C y D), respuesta correcta, justificación, bibliografía, competencia, tema, subtema y nivel de dificultad.
- Al darle «Guardar», el sistema revisa la pregunta. Si falta algo o algo está mal, muestra todos los errores juntos y no la guarda.
- El contexto, la pregunta y la justificación deben tener mínimo 10 caracteres, y el título máximo 100.
- Las cuatro opciones no se pueden repetir, la respuesta correcta tiene que ser A, B, C o D, y la dificultad tiene que ser Fácil, Media o Difícil.
- Si todo está bien, la pregunta se guarda en estado «Borrador» con un código consecutivo (P-001, P-002, …).

2.2.- HU-02: Enviar pregunta a revisión

- En «Mis preguntas» el autor selecciona una pregunta en «Borrador» y le da «Enviar a revisión».
- El sistema pide confirmación, porque después de enviarla ya no se puede editar.

- La pregunta pasa a estado «Pendiente de revisión».
- No se puede enviar una pregunta que no esté en «Borrador» ni una pregunta de otro autor.
- Cada estado se ve con un color distinto en la tabla.

2.3.- HU-03: Listar mis preguntas

- Solo aparecen las preguntas del autor que inició sesión.
- Se puede filtrar por estado, dificultad, tema, competencia y por un texto que se busca en el título, el contexto y la pregunta.
- La tabla se muestra por páginas de 5, 10 o 20 filas, con los botones «Anterior» y «Siguiente».
- Si la pregunta está en «Borrador» se puede editar; si no, solo se puede ver.

2.4.- HU-04: Asignar revisores

- El administrador ve las preguntas en «Pendiente de revisión» y la lista de revisores activos.
- Puede escoger uno o varios revisores (con la tecla Ctrl) y asignarlos.
- No se puede asignar como revisor al autor de la pregunta, ni a usuarios inactivos o que no tengan el rol Revisor.
- Si un revisor ya estaba asignado a esa pregunta, no se repite.
- A cada revisor nuevo le llega un correo. Si no existe el archivo mail.properties, el correo se muestra en la consola.
- Si un correo falla, la asignación igual queda guardada y el sistema avisa cuál falló.

3.- Prototipos y prueba de usabilidad

3.1.- Prototipos

Antes de programar hicimos bocetos (wireframes) de las ventanas principales. Después las construimos con Java Swing, así que las ventanas reales se parecen mucho a estos bocetos.

La Figura 3.1 es el formulario para crear una pregunta (HU-01). La respuesta correcta y la dificultad se escogen de una lista para que el autor no se equivoque al escribirlas.

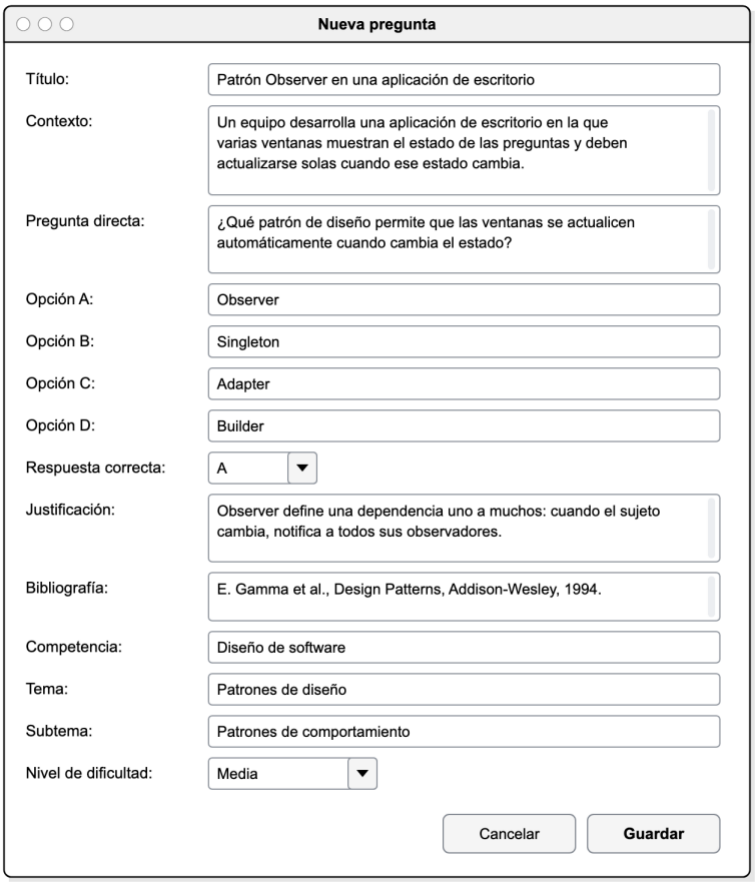


Figura 3.1 Prototipo del formulario para crear una pregunta

La Figura 3.2 es la ventana «Mis preguntas» (HU-02 y HU-03). Arriba están los filtros, en el centro la tabla y abajo los botones y la paginación.

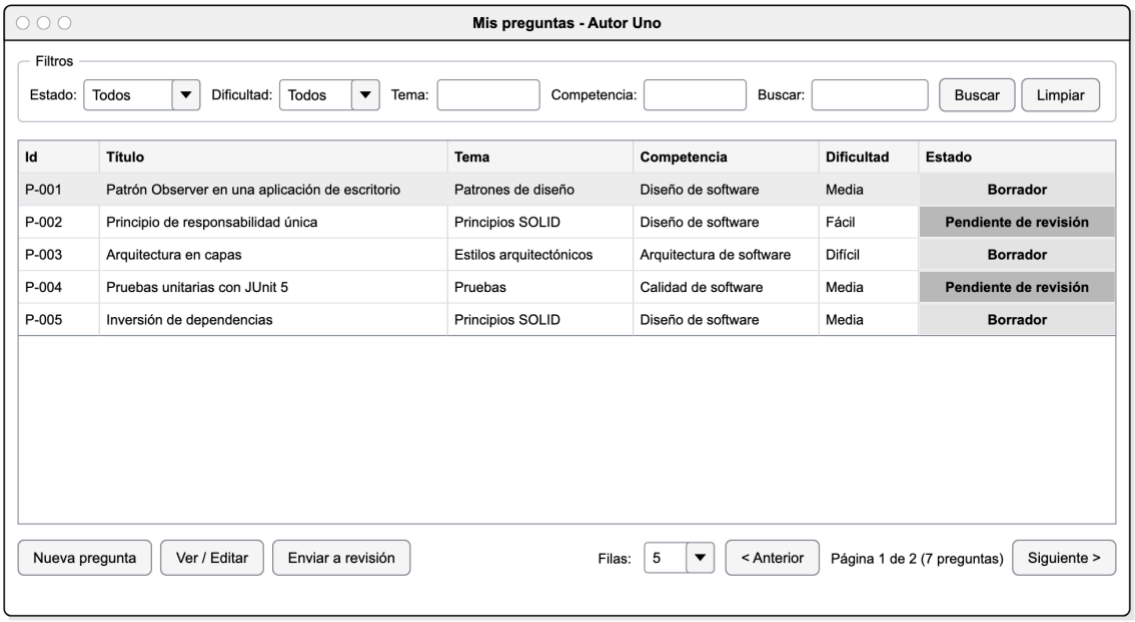


Figura 3.2 Prototipo de la ventana «Mis preguntas»

La Figura 3.3 es la ventana del administrador para asignar revisores (HU-04).

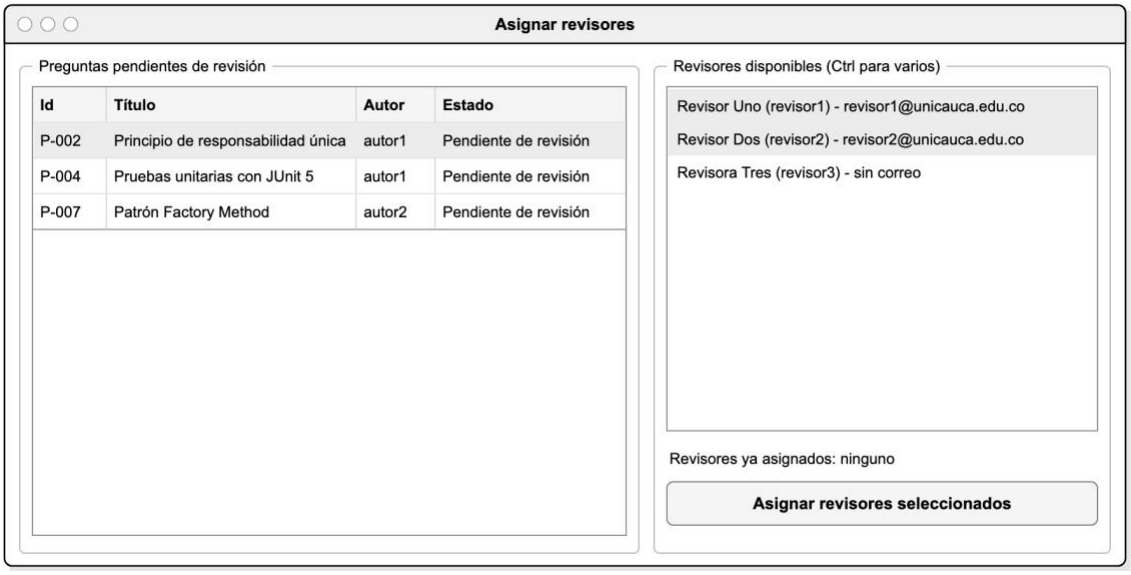


Figura 3.3 Prototipo de la ventana «Asignar revisores»

3.2.- Prueba de usabilidad

Para probar si las ventanas se entienden usamos el método *pensar en voz alta* (thinking aloud): le pedimos a una persona que no conocía el programa que hiciera unas tareas mientras decía en voz alta lo que iba pensando. Nosotros solo observábamos y anotábamos dónde se confundía.

Participantes y fecha: [Completar].

Tarea	¿La logró?	Observaciones
Iniciar sesión como autor.	[Completar]	[Completar]
Crear una pregunta completa.	[Completar]	[Completar]
Buscar una pregunta usando los filtros.	[Completar]	[Completar]
Enviar una pregunta a revisión.	[Completar]	[Completar]
Iniciar sesión como administrador y asignar un revisor.	[Completar]	[Completar]

Tabla 3.1 Tareas de la prueba de usabilidad

Lo que encontramos y cambiamos: [Completar].

4.- Planificación del Sprint 1

Organizamos el trabajo como un sprint de Scrum y repartimos las tareas en un tablero. La Figura 4.1 muestra el tablero y la Tabla 4.1 las tareas principales.



Figura 4.1 Tablero de tareas del Sprint 1

Tarea	Responsable
Hacer los bocetos de las ventanas.	[Completar]
Crear la pregunta con validación (HU-01).	[Completar]
Enviar a revisión y colores de los estados (HU-02).	[Completar]
Listado con filtros y paginación (HU-03).	[Completar]
Asignar revisores y enviar correos (HU-04).	[Completar]
Pruebas unitarias.	[Completar]
Documento y video.	[Completar]

Tabla 4.1 Tareas del Sprint 1

5.- Escenario de calidad: modificabilidad

El atributo de calidad más importante en este corte es la modificabilidad, o sea, que el programa se pueda cambiar fácilmente sin dañar lo que ya funciona. Lo describimos con un escenario de calidad (Tabla 5.1).

Parte	Descripción
Contexto	Hoy los revisores reciben un correo cuando se les asigna una pregunta.
Estímulo	El cliente pide que también se pueda avisar por otro medio, por ejemplo WhatsApp o SMS.
Artefacto	La parte de notificaciones (paquete infra) y la clase ReviewService.
Respuesta	Se crea una clase nueva que implemente la interfaz INotificador y se agrega en NotificadorFactory. No hay que tocar ReviewService, ni los controladores, ni las ventanas.
Medida	Máximo dos clases tocadas (una nueva y la fábrica), menos de 4 horas de trabajo y todas las pruebas que ya existen siguen pasando.
Resultado esperado	El cambio queda aislado gracias a la interfaz INotificador y a la fábrica.

Tabla 5.1 Escenario de calidad de modificabilidad

6.- Arquitectura

6.1.- Tipo de aplicación y tecnologías

Hicimos una aplicación de escritorio porque así lo pedía el primer corte. Esto tiene la ventaja de que no se necesita un servidor, pero la desventaja es que cada computador tiene su propia base de datos. En el segundo corte el sistema pasará a microservicios. La Tabla 6.1 muestra las tecnologías que usamos.

Tecnología	Para qué la usamos
Java 17	Lenguaje del proyecto; es el que sugiere el curso.
Java Swing	Las ventanas de la aplicación. Ya viene con Java, no hay que instalar nada más.
Maven	Compilar, descargar las librerías y correr las pruebas.
SQLite (sqlite-jdbc)	Base de datos en un solo archivo (users.db). No necesita servidor.
Argon2 (argon2-jvm)	Guardar las contraseñas con hash, nunca en texto plano.
Jakarta Mail	Enviar los correos a los revisores.
JUnit 5	Pruebas unitarias automáticas.
Git y GitHub	Control de versiones y trabajo en equipo.

Tabla 6.1 Tecnologías usadas

6.2.- Diagramas C4

Usamos el modelo C4 [3], que explica la arquitectura por niveles, como si uno hiciera zoom. El nivel 1 (Figura 6.1) muestra quién usa el sistema y con qué sistemas externos se comunica.

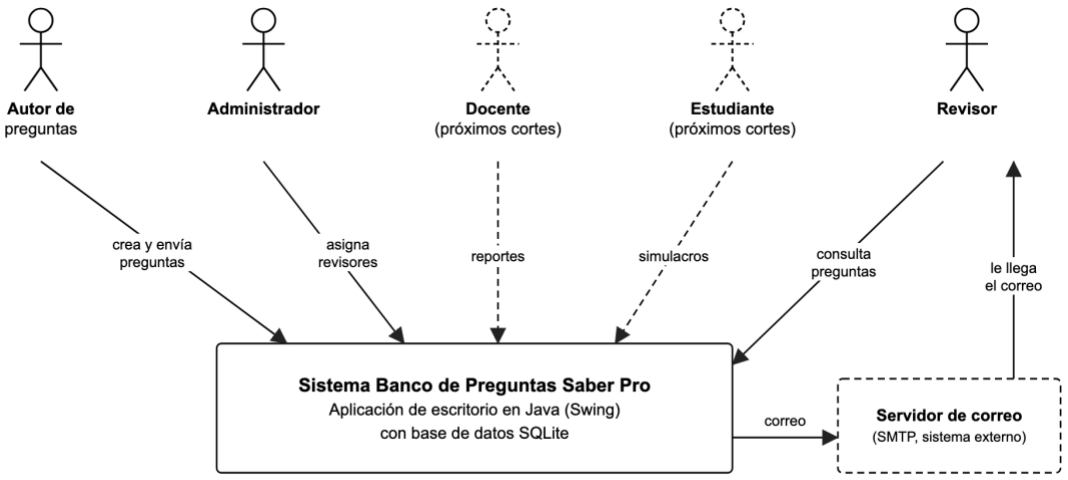


Figura 6.1 Diagrama de contexto (C4 nivel 1)

En el nivel 2 (contenedores) el sistema tiene tres partes:

- La aplicación de escritorio en Java.
- La base de datos SQLite (archivo users.db).
- Dos archivos de configuración: plugins.properties y mail.properties, que tiene los datos del correo y no se sube a GitHub.

El nivel 3 (Figura 6.2) muestra las clases más importantes organizadas por capas.

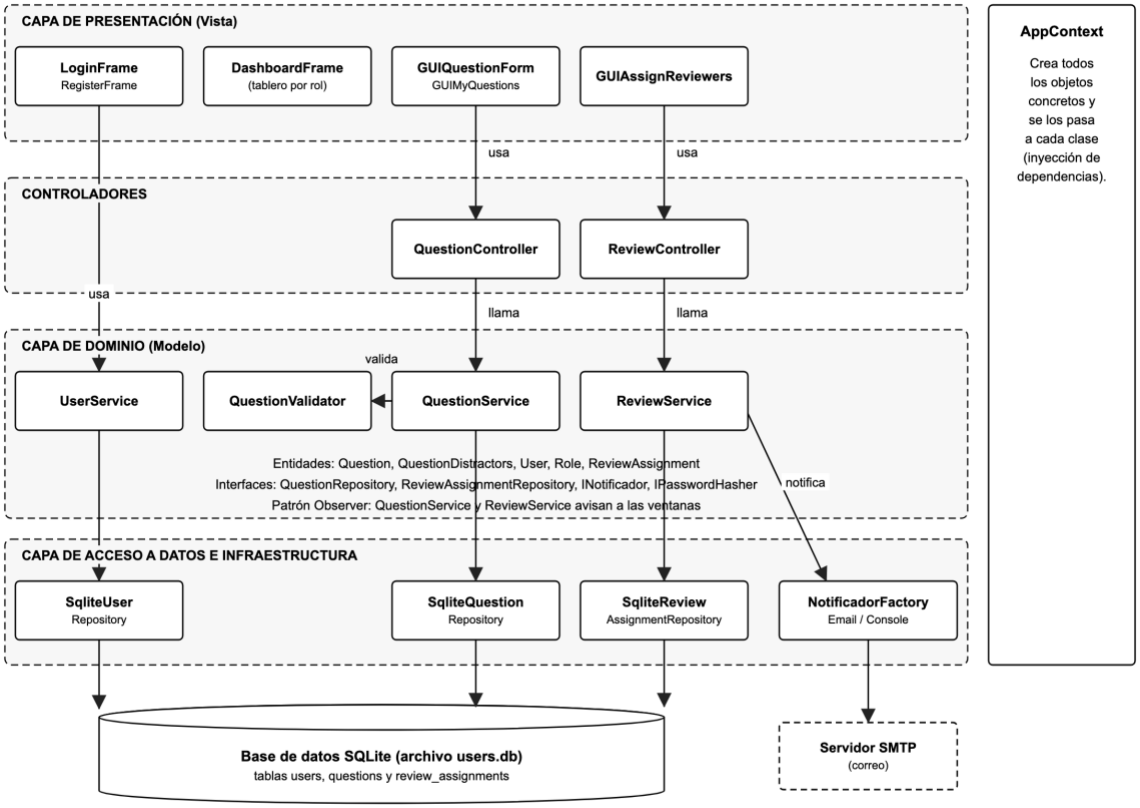


Figura 6.2 Diagrama de componentes por capas (C4 nivel 3)

6.3.- ¿Por qué capas y MVC?

El programa está dividido en tres capas y cada una solo habla con la de abajo:

1. **Presentación:** las ventanas (vistas) y los controladores.
2. **Dominio:** las reglas del negocio, por ejemplo qué hace válida a una pregunta.
3. **Acceso a datos e infraestructura:** guardar en SQLite y enviar correos.

Dentro de la presentación usamos **MVC**:

- La **vista** es la ventana de Swing.
- El **controlador** recibe lo que hace el usuario y llama al servicio.
- El **modelo** son las clases del dominio.

Así, si mañana cambiamos Swing por una página web, la lógica de las preguntas no cambia. También es más fácil hacer pruebas, porque los servicios se pueden probar sin abrir ventanas.

7.- Explicación del código

7.1.- Organización de los paquetes

Todo el código está en el paquete co.edu.unicauca. La Tabla 7.1 explica qué hay en cada subpaquete.

Paquete	Qué contiene
presentation	Las ventanas Swing: LoginFrame, DashboardFrame, GUIQuestionForm, GUIMyQuestions, GUIAssignReviewers y las ventanas del banco de preguntas.
controller	QuestionController y ReviewController.
domain	Las entidades (Question, User, ReviewAssignment…), los servicios (QuestionService, ReviewService, UserService), los validadores y las interfaces de repositorio.
access	Las clases que leen y escriben en SQLite (SqliteQuestionRepository, SqliteUserRepository, SqliteReviewAssignmentRepository).
infra	El patrón Observer (Subject, Observer) y los notificadores de correo.
kernel, plugin, plugins, pipeline	El generador de preguntas con plugins que hicimos en el taller 5.

Tabla 7.1 Paquetes del proyecto

7.2.- ¿Cómo arranca la aplicación?

La clase `Main` crea un `AppContext` y abre la ventana de inicio de sesión. `AppContext` es la clase que crea todos los objetos y los conecta entre sí. Por ejemplo, a `ReviewService` le entrega el repositorio y el notificador que debe usar:

```
public AppContext() {
    SqliteDatabase db = new SqliteDatabase();
    IUserRepository userRepository =
            new SqliteUserRepository(SqliteDatabase.DEFAULT_URL);
    QuestionRepository questionRepository = new SqliteQuestionRepository(db);
    ReviewAssignmentRepository assignmentRepository =
            new SqliteReviewAssignmentRepository(db);
    INotificador notificador = NotificadorFactory.crear();
    IPasswordHasher hasher = new PasswordHasher();
    IPasswordValidator validator = new PasswordValidator();

    this.userService = new UserService(userRepository, hasher, validator);
    this.questionService = new QuestionService(questionRepository);
    this.reviewService = new ReviewService(questionRepository,
            assignmentRepository, userRepository, notificador);
    ...
}
```

Como los servicios reciben interfaces y no clases concretas, si cambiamos SQLite por otra base de datos solo hay que cambiar esta clase. Además, si la base de datos está vacía, `AppContext` crea cinco usuarios de prueba (admin, autor1, revisor1, docente1 y estudiante1).

7.3.- Clases del dominio

- `Question`: una pregunta con todos sus campos (contexto, pregunta, opciones, respuesta, justificación, competencia, etc.), su estado y su autor.
- `QuestionDistractors`: las cuatro opciones A, B, C y D de la pregunta.
- `User`: un usuario con login, nombre, correo, contraseña (en hash), rol y estado.
- `Role` y `UserStatus`: enumeraciones con los roles (Administrador, Autor de preguntas, Revisor, Docente y Estudiante) y los estados (Activo e Inactivo).
- `ReviewAssignment`: la asignación de un revisor a una pregunta, con quién la asignó y la fecha.

7.4.- HU-01: crear una pregunta paso a paso

Cuando el autor le da «Guardar» en `GUIQuestionForm`, la ventana lee los campos, arma un objeto `Question` y se lo pasa al controlador. El controlador le pone el autor que inició sesión:

```
public Question crear(Question question) {
    question.setAutor(author.getLogin());
    return service.createQuestion(question);
}
```

Luego `QuestionService` valida la pregunta, le pone el código y el estado «Borrador», la guarda y avisa a las ventanas que haya algo nuevo:

```
public Question createQuestion(Question question) {
    if (question == null) {
        throw new IllegalArgumentException("La pregunta no puede ser nula.");
    }
    validator.validateOrThrow(question);
    question.setId(nextId());
    question.setEstado(Question.STATE_BORRADOR);
    if (!repository.save(question)) {
        throw new IllegalStateException("No se pudo guardar la pregunta.");
    }
    notifyAllObservers();
    return question;
}
```

La validación está en `QuestionValidator`. Esta clase va acumulando todos los errores en una lista para mostrarlos juntos. Por ejemplo, así revisa que las opciones no estén vacías ni repetidas:

```
String[] opciones = {d.getOptionA(), d.getOptionB(), d.getOptionC(), d.getOptionD()};
Set<String> vistas = new HashSet<>();
for (int i = 0; i < opciones.length; i++) {
    char letra = (char) ('A' + i);
    if (esVacio(opciones[i])) {
        errores.add("La opción " + letra + " es obligatoria.");
    } else if (!vistas.add(opciones[i].trim().toLowerCase())) {
        errores.add("La opción " + letra + " está repetida.");
    }
}
```

Si la lista tiene errores, `validateOrThrow` lanza una excepción con todos los mensajes y la ventana los muestra. Si no hay errores, `SqliteQuestionRepository.save` guarda la pregunta con un `PreparedStatement`, que además evita la inyección SQL.

7.5.- HU-02: enviar a revisión

En `QuestionService` primero se busca la pregunta y se revisa que sea del autor. Después se verifica que esté en «Borrador» y se cambia el estado:

```
public void sendToReview(String questionId, String authorLogin) {
    Question q = getOwnedQuestion(questionId, authorLogin);
    if (!Question.STATE_BORRADOR.equals(q.getEstado())) {
        throw new IllegalStateException(
                "Solo se pueden enviar a revisión preguntas en estado Borrador.");
    }
    repository.updateState(questionId, Question.STATE_PENDIENTE_REVISION);
    notifyAllObservers();
}
```

Los colores de cada estado están en la clase `EstadoColores`: azul para «Borrador», ámbar para «Pendiente de revisión» y rojo para «Eliminada».

7.6.- HU-03: filtros y paginación

La clase `QuestionFilter` guarda los filtros que el usuario escribió. Su método `matches` dice si una pregunta cumple todos los filtros (los que estén vacíos se ignoran). Después, `searchByAuthor` calcula qué preguntas van en la página pedida:

```
int total = filtered.size();
int totalPages = Math.max(1, (int) Math.ceil(total / (double) pageSize));
int currentPage = Math.min(Math.max(page, 1), totalPages);
int from = (currentPage - 1) * pageSize;
int to = Math.min(from + pageSize, total);
List<Question> items = new ArrayList<>(filtered.subList(from, to));
```

El resultado se devuelve en un `PageResult`, que trae la lista de la página y los datos para los botones «Anterior» y «Siguiente».

7.7.- HU-04: asignar revisores

`ReviewService.assignReviewers` hace las validaciones en dos pasos:

1. Revisa que quien asigna sea administrador y que la pregunta esté «Pendiente de revisión».
2. Revisa a todos los revisores escogidos (que existan, tengan el rol correcto, estén activos y no sean el autor). Si uno falla, no se guarda ninguno.

Al final guarda cada asignación nueva y le avisa al revisor:

```
List<String> fallos = new ArrayList<>();
for (User revisor : nuevos) {
    assignmentRepository.save(new ReviewAssignment(questionId, revisor.getLogin(),
            admin.getLogin(), LocalDateTime.now()));
    try {
        notificador.notificarAsignacion(revisor, question);
    } catch (NotificacionException e) {
        fallos.add(revisor.getLogin() + ": " + e.getMessage());
    }
}
notifyAllObservers();
return fallos;
```

Si un correo falla, el error se guarda en la lista `fallos` y la asignación no se pierde. Como enviar correos puede demorar, la ventana `GUIAssignReviewers` hace esta llamada con un `SwingWorker` para que la pantalla no se congele.

7.8.- Base de datos

Usamos SQLite y cada repositorio crea su tabla al iniciar (`CREATE TABLE IF NOT EXISTS`). La Tabla 7.2 muestra las tres tablas que tiene la base de datos en este corte.

Tabla	Columnas principales	Para qué sirve
users	login (llave primaria), full_name, role, status, password_hash, email	Usuarios y sus roles.

Tabla	Columnas principales	Para qué sirve
questions	id (llave primaria), nombre, contexto, pregunta, option_a a option_d, respuesta_correcta, justificacion, bibliografia, competencia, tema, subtema, dificultad, estado, autor	Las preguntas del banco.
review_assignments	question_id y reviewer_login (llave primaria compuesta), assigned_by, assigned_at	Qué revisor tiene cada pregunta.

Tabla 7.2 Tablas de la base de datos

7.9.- Contraseñas

Cuando alguien se registra, `PasswordValidator` revisa que la contraseña tenga mínimo 6 caracteres, una mayúscula, un número y un carácter especial. Después `PasswordHasher` la convierte en un hash con Argon2id, y eso es lo que se guarda. Al iniciar sesión se compara el hash, nunca la contraseña en texto plano.

8.- Principios SOLID y patrones de diseño

8.1.- Principios SOLID

- **S, responsabilidad única:** cada clase hace una sola cosa. Por ejemplo, `QuestionValidator` solo valida y `PasswordHasher` solo genera el hash.
- **O, abierto/cerrado:** podemos agregar un notificador nuevo o un tipo de pregunta nuevo (plugin) sin modificar las clases que ya existen.
- **L, sustitución de Liskov:** `EmailNotificador` y `ConsoleNotificador` se pueden usar uno en lugar del otro, porque los dos cumplen `INotificador`. En las pruebas usamos clases falsas (`FakeUserRepository`, `FakeNotificador`) y los servicios funcionan igual.
- **I, segregación de interfaces:** las interfaces son pequeñas. `INotificador` tiene un solo método y `IPasswordHasher` tiene dos.
- **D, inversión de dependencias:** los servicios dependen de interfaces que reciben en el constructor, y `AppContext` es el único que crea los objetos concretos.

8.2.- Patrones de diseño

La Tabla 8.1 resume los patrones que usamos.

Patrón	Dónde está	Para qué lo usamos
MVC	Ventanas, controladores y servicios	Separar la interfaz de la lógica.
Observer	Subject, Observer, QuestionService, ReviewService y las ventanas	Que las ventanas se actualicen solas cuando cambia una pregunta.
Factory	NotificadorFactory	Decidir si los correos se envían de verdad o se muestran en consola.
Singleton	QuestionPluginManager	Tener un solo administrador de plugins en toda la aplicación.
Repository	QuestionRepository y sus implementaciones	Separar el dominio de la base de datos.
Microkernel y tuberías y filtros	kernel, plugins y pipeline	Agregar tipos de pregunta como plugins y validarlos con filtros (taller 5).

Tabla 8.1 Patrones de diseño usados

8.2.1.- Observer

`QuestionService` y `ReviewService` heredan de `Subject`. Cada vez que algo cambia llaman a `notifyAllObservers()`:

```
public void notifyAllObservers() {
    for (Observer observer : observers) {
        observer.update(this);
    }
}
```

Las ventanas como `GUIMyQuestions` implementan `Observer`. Cuando les llega el aviso, vuelven a cargar la tabla:

```
@Override
public void update(Object obj) {
    cargar();
}
```

8.2.2.- Factory

`NotificadorFactory` revisa si existe el archivo mail.properties. Si no existe (o le faltan datos), devuelve un `ConsoleNotificador` que muestra el correo en la consola; si todo está bien, devuelve un `EmailNotificador` que envía el correo de verdad:

```
public static INotificador crear(Path configFile) {
    if (!Files.exists(configFile)) {
        System.out.println("No se encontró " + configFile
                + ": los correos se simularán por consola.");
        return new ConsoleNotificador();
    }
    ...
    return new EmailNotificador(props);
}
```

9.- Pruebas unitarias

Hicimos las pruebas con JUnit 5 y se corren con el comando `mvn test`. En total hay 18 clases de prueba con 110 pruebas. Para probar los servicios no usamos la base de datos real, sino repositorios falsos en memoria; así las pruebas son rápidas y no dependen de nada externo. Este es un ejemplo:

```
@Test
@DisplayName("HU1: crea la pregunta en estado Borrador con id nuevo")
void crearPregunta() {
    Question creada = service.createQuestion(TestData.preguntaValida("autor1"));
    // el repositorio de ejemplo ya trae P-001..P-005
    assertEquals("P-006", creada.getId());
    assertEquals(Question.STATE_BORRADOR, creada.getEstado());
    assertNotNull(service.getQuestionById("P-006"));
}
```

Grupo	Clases de prueba	Pruebas
Servicios	QuestionServiceHistoriasTest, ReviewServiceTest, QuestionServiceTest, UserServiceTest	48
Validaciones	QuestionValidatorTest, PasswordValidatorTest, QuestionFilterTest, PageResultTest	22
Repositorios	SqliteQuestionRepositoryTest, SqliteUserRepositoryTest, SqliteReviewAssignmentRepositoryTest, QuestionImplRepositoryTest	21
Entidades	QuestionTest, UserTest	8
Otras	ControllersTest, ObserverPatternTest, PasswordHasherTest, MultipleChoicePipelineTest	11
Total		**110**

Tabla 9.1 Resumen de las pruebas unitarias

Pantallazo de la ejecución de las pruebas: [Completar].

10.- Lo que falta y lo que vamos a mejorar

- Todavía no validamos RF-12 (no permitir opciones como «Todas las anteriores») ni RF-13 (longitud y gramática de los distractores). Los vamos a agregar en `QuestionValidator`.
- Los estados de la pregunta son textos (String). Sería mejor usar un enum para controlar qué cambios de estado son válidos.
- La interfaz `IUserRepository` está en el paquete access y debería estar en domain, como las otras interfaces de repositorio.
- En el segundo corte la aplicación se va a dividir en microservicios, y la base de datos pasará a PostgreSQL o MariaDB.

11.- Enlaces

- **Video en YouTube:** [Completar con el enlace]
- **Repositorio en GitHub:** [Completar con el enlace]

Para ejecutar el proyecto:

- Compilar: `mvn compile`
- Correr las pruebas: `mvn test`
- Abrir la aplicación: `mvn exec:java`

Los usuarios de prueba y sus contraseñas están en el README del repositorio. Para que los correos se envíen de verdad hay que copiar mail.properties.example como mail.properties y llenar los datos del correo.

Bibliografía

[1] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, EE. UU.: Addison-Wesley, 1994.

[2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, EE. UU.: Prentice Hall, 2017.

[3] S. Brown, "The C4 model for visualising software architecture". [En línea]. Disponible en: https://c4model.com

[4] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 4.ª ed. Boston, MA, EE. UU.: Addison-Wesley, 2021.

[5] Universidad del Cauca, "Proyecto de clase 2026.2: Sistema para la gestión, validación y administración de un banco de preguntas para la preparación de las pruebas Saber Pro," Ingeniería de Software II, 2026.